

**KWAME NKRUMAH UNIVERSITY OF SCIENCE AND
TECHNOLOGY**

COLLEGE OF SCIENCE

FACULTY OF PHYSICAL AND COMPUTATIONAL SCIENCE

DEPARTMENT OF COMPUTER SCIENCE



PROJECT TOPIC:

**DERA - A YOUTH EMPOWERMENT PLATFORM FOR AT RISK AND
UNDERSERVED GHANAIAN YOUTH**

SUBMITTED BY

VERONICA NAKOL NGMALBINI - 9029123

PROJECT SUPERVISOR - DR. KWABENA OWUSU-AGYEMANG

2025 / 2026 ACADEMIC YEAR

DECLARATION

I hereby declare that this project report is my own original work, undertaken as partial fulfilment of the requirement for the award of a Bachelor of Science degree in Computer Science at the Kwame Nkrumah University of Science and Technology, and that, to the best of my knowledge, it contains no material previously published by another person, nor material which has been accepted for the award of any other degree, except where due acknowledgement has been made in the text.

DEDICATION

This work is dedicated to the young people across Ghana's underserved communities whose stories motivated this project, and to our families, whose support made the journey possible.

ACKNOWLEDGEMENT

To God be the Glory.

I wish to express my sincere gratitude to my project supervisor for the guidance, patience, and constructive criticism that shaped this work from a rough idea into a working platform. I am also grateful to the counsellors, teachers, and youth facing NGO staff who shared their experiences and helped validate the requirements behind DERA, and to my families and friends for their unwavering support and encouragement throughout this project.

ABSTRACT

DERA (“Every Young Person Belongs Here”) is a role-based web platform, delivered as an installable Progressive Web App, designed to support at-risk and underserved youth in Ghana with career discovery, scholarship access, mentorship, self-development resources, and donor-funded support, while giving school and NGO counsellors the tools to monitor and intervene on early warning signs of dropout. The platform serves four kinds of users - youth, counsellors, donors, and administrators - through a single Django REST Framework backend and a shared React frontend whose navigation, permissions, and dashboards adapt to the signed-in role. Distinguishing features include an open-ended, adaptive AI Career Counsellor that replaces the traditional fixed-choice career quiz with a natural conversation; an anonymous AI companion, Auntie DERA, that lets youth discuss sensitive topics such as re-enrolment, family pressure, and reproductive health without ever being linked to their identity; a data-driven risk-assessment module that lets counsellors log academic, attendance, and behavioural indicators and track interventions; a transparent marketplace that lets donors sponsor a specific youth's application form or a specific community project and see the resulting impact reports; and a curated self-development library with an AI Growth Librarian that recommends real books by category and reading level. The system was built with Django 6 and Django REST Framework on PostgreSQL (via Supabase) for

the backend, and React 19 with Vite, Tailwind CSS, and React Router for the frontend, integrating Google Gemini and Groq-hosted large language models for its conversational features. This report documents the background, requirements, architecture, implementation, and testing of DERA, and concludes with the lessons learnt and recommendations for scaling the platform beyond its pilot scope.

TABLE OF CONTENTS

DECLARATION	2
DEDICATION	2
ACKNOWLEDGEMENT	2
ABSTRACT	2
CHAPTER ONE	7
1.0 Introduction	7
1.1 Background of Project	7
1.2 Problem Statement	7
1.3 Aim of the Project	8
1.4 Novelty of the Project	8
1.5 Specific Objectives of the Project	9
1.6 Scope of the Project	9
1.7 Project Limitations	10
Technical Limitations	10
Adoption and Resource Limitations	10
Anonymity Trade-offs	10
1.8 Academic and Practical Relevance of the Project	10
1.8.1 Academic Relevance	10
1.8.2 Practical Relevance	11
1.9 Beneficiaries of the Project	11
1.10 Project Activity Planning and Schedules	11
1.11 Definitions and Explanations of Terms	12
1.12 Structure of Report	13
CHAPTER TWO	14
2.1 Review of System 1: MTN Ghana Foundation Bright Scholarship Portal (scholarship.mtn.com.gh)	14
Description of System	14
Review of the Good Features	14
Review of the Bad Features	14
Summary of the System Review	15
2.2 Review of System 2: Chancen International Income Share Agreement Platform	15
Description of System	15
Review of the Good Features	15

Review of the Bad Features	15
Summary of the System Review	16
2.3 Review of System 3: Zeraki Analytics	16
Description of System.....	16
Review of the Good Features.....	16
Review of the Bad Features	16
Summary of the System Review	16
2.4 Review of System 4: Worldreader Mobile	17
Description of System.....	17
Review of the Good Features.....	17
Review of the Bad Features	17
Summary of the System Review	17
2.5 Review of System 5: CareerExplorer (Sokanu).....	17
Description of System.....	17
Review of the Good Features.....	18
Review of the Bad Features	18
Summary of the System Review	18
2.6 Conceptual Design of the Proposed Project.....	18
CHAPTER THREE	20
3.1 Introduction.....	20
3.2 The Architecture of the Proposed Project	20
3.3 Requirements Elicitation Process of the Proposed Project	21
3.4 Functional Requirements of the Proposed Project.....	21
3.5 NonFunctional Requirements of the Proposed Project	22
3.6 UML Diagrams	22
3.6.1 Use Case Diagram - FrontEnd (Youth-facing) Models	22
3.6.2 Use Case Diagram - BackEnd (Counsellor, Donor, Admin) Models	23
3.7 Activity Diagrams of the System.....	26
Youth Onboarding and Career Discovery Flow	26
Risk Assessment and Intervention Flow	28
3.8 Sequence Diagrams of the System.....	30
AI Career Counsellor Conversation Sequence	30
Scholarship Application Form Sponsorship Sequence	30
3.9 Class Diagrams of the System	31

3.10 Users of the Proposed System.....	32
3.10.1 User Characteristics	32
3.11 Security Concepts of the System	32
3.12 Project Method Employed	33
3.13 Software Process Models Employed and Justification of the Chosen Model	33
3.14 Chosen Model and Justification.....	33
3.15 Project Design Considerations: Logical Designs.....	34
3.15.1 UI Design (Wireframes)	34
3.15.2 DB Design (DB Schemas)	37
CHAPTER FOUR.....	39
4.1 Introduction.....	39
4.2 Mapping Logical Design onto Physical Platform.....	39
4.3 System Modules Implementation	40
4.4 System Modules Integration	41
4.4.1 Testing.....	41
4.4.2 Testing Plan	42
4.4.3 Verification Testing	42
4.4.4 Validation Testing.....	42
4.4.5 System Security Testing	42
4.4.6 Recommendations Made by Testers	43
4.4.7 Responses to Recommendations from Testing.....	43
4.5 Results.....	43
CHAPTER FIVE	44
5.1 Introduction.....	44
5.2 Findings.....	44
5.3 Conclusions.....	44
5.4 Challenges/Limitations of the System	44
5.5 Lesson Learnt.....	45
5.6 Recommendations for Future Works	45
5.7 References.....	45
APPENDICES	46
Appendix A: Sample Test Cases.....	46
Appendix B: Selected Environment / Configuration Reference.....	46

CHAPTER ONE

INTRODUCTION

1.0 Introduction

DERA is a youth empowerment platform built to close the gap between underserved Ghanaian youth and the career guidance, scholarship information, mentorship, and safety-net support that already exist but are scattered across schools, NGOs, telecom foundations, and government agencies. The rest of this chapter covers the background and problem the project responds to, its aim and specific objectives, the novelty it introduces, its scope and limitations, its academic and practical relevance, its beneficiaries, the plan followed to deliver it, a glossary of terms used throughout the report, and an outline of the chapters that follow.

1.1 Background of Project

Ghana has made steady progress in basic and secondary school enrolment over the last two decades, but large numbers of young people, particularly in rural and peri-urban communities, still fall out of the education pipeline before they reach a career of their choosing. The reasons are rarely a single event: a mix of financial hardship, unplanned pregnancy, family pressure to work instead of study, undiagnosed academic decline, and a simple lack of accurate, timely information about the scholarships, courses, and careers open to them. Where support does exist - corporate scholarship schemes, NGO mentorship programmes, school counselling offices, community donors willing to fund a promising student - it is fragmented across phone calls, paper forms, WhatsApp groups, and one-off outreach visits, with no shared record of who has already been helped or who is falling behind.

At the same time, smartphone penetration and mobile data access among Ghanaian youth have grown quickly enough that a mobile-first digital platform is now a realistic channel for reaching this population directly, rather than only through the adults around them. DERA was conceived against this backdrop: a single platform that gives youth themselves a self-service way to discover careers and scholarships, gives counsellors a shared system to track and act on risk indicators for the youth assigned to them, and gives donors a transparent, need-specific way to fund a young person's next step, instead of a generic and opaque donation.

1.2 Problem Statement

Youth in under-resourced Ghanaian communities face four compounding problems that DERA addresses directly:

1. Information asymmetry: youth are frequently unaware of the scholarships, application deadlines, and career pathways that already exist for people in their situation, because that information is not centralised or written for them.

2. Fixed, one-off career guidance: where career guidance is available at all, it is usually a single generic quiz or a onetime talk, rather than an ongoing, adaptive conversation that responds to an individual's actual interests, strengths and circumstances.
3. Invisible early warning signs: counsellors and teachers often only learn that a youth is at risk of dropping out after attendance or grades have already collapsed, because academic, attendance, and behavioural indicators are not tracked systematically in one place.
4. Opaque and inaccessible funding: donors who want to support a specific young person or project have few channels that let them see exactly what their contribution paid for, while youth who need help - for example, paying an application form fee - have no structured way to request sponsorship.

Existing responses to these problems - corporate scholarship portals, school counselling offices, NGO helplines, and community fundraising - solve one piece each, but none combine discovery, guidance, risk monitoring, and funding into a single system built around the youth.

1.3 Aim of the Project

The aim of the DERA project is to design, develop, and deploy a role-based web platform that gives underserved Ghanaian youth self-service access to career discovery, scholarship information, mentorship, and self-development resources; gives school and NGO counsellors tools to monitor risk indicators and coordinate interventions for the youth assigned to them; and gives donors a transparent channel to fund specific youth needs and community projects, all through a single shared system.

1.4 Novelty of the Project

DERA's novelty lies less in any single feature than in the combination of the following, applied specifically to the Ghanaian youth-support context:

- An adaptive, conversational AI Career Counsellor that asks one open-ended question at a time and shapes each next question around what the youth has already said, rather than presenting a fixed multiple-choice quiz, so that no two youths are guided through an identical script.
- Auntie DERA, an anonymous AI companion for the Help Centre, built around a deliberate architectural guarantee: the HelpRequest data model carries no foreign key to any user account, so a youth can raise a sensitive topic - re-enrolment after a break, family pressure, or reproductive health - with no way for their identity to ever be attached to that record.
- A structured risk-assessment pipeline that turns academic records, attendance, and counsellor observations into a scored RiskAssessment with categorised indicators (academic decline, absenteeism, financial distress, behavioural change) and trackable interventions, rather than relying on a counsellor's memory.
- A need-specific donor marketplace in which a donor can sponsor a named youth's application form fee or a named community project, and later see the impact report linked to that same project, rather than giving into an undifferentiated fund.

- An AI Growth Librarian that curates real, hand-selected books (never invented titles) across sixteen categories such as financial literacy, entrepreneurship, and academic success, matched to a youth's stated goals, alongside a personal reading tracker.
- Delivery as an installable, offline-tolerant Progressive Web App, so that intermittent connectivity in rural areas degrades the experience gracefully instead of breaking it.

1.5 Specific Objectives of the Project

1. To design and implement a single, role-based platform (youth, counsellor, donor, admin) with authentication and dashboards tailored to each role.
2. To build a career-discovery flow combining an AI-driven conversational counsellor with a structured catalogue of career paths, careers, and courses linked to real institutions.
3. To provide youth with a searchable index of scholarships and opportunities, including deadline-aware notifications.
4. To give counsellors a risk-assessment and intervention-tracking module for the youth assigned to them, alongside a counselling-session scheduler.
5. To build an anonymous Help Centre channel and an AI companion for sensitive topics that structurally cannot be traced back to a user.
6. To provide a transparent donation and application-form marketplace that lets donors fund specific youth needs and projects and view impact reports.
7. To curate a self-development library with an AI-assisted recommendation engine and a personal reading tracker.
8. To evaluate the resulting system through unit, integration, and user acceptance testing, and refine it based on the findings.

1.6 Scope of the Project

The scope of DERA covers the following:

1. Design and development of a responsive, installable web application (Progressive Web App) usable on both desktop and mobile browsers.
2. Four user roles - Youth, Counselor, Donor, and Admin - each with its own dashboard, navigation, and permissions enforced at the API layer.
3. Eleven backend modules: accounts, risk, careers, marketplace, stories, helpcentre, mentorship, learning, donations, search, and library, exposed through a single Django REST Framework API.
4. Integration with two hosted large-language-model providers (Google Gemini and Groq) for the AI Career Counsellor, Auntie DERA Help Centre companion, and AI Growth Librarian.
5. A curated seed dataset covering Ghanaian senior high schools, universities, technical universities, colleges of education, and nursing training colleges, together with sample scholarships, career paths, and books.
6. A donation and application-form purchase flow, including a payment record and sponsorship model, built to a state ready for integration with a live mobile-money gateway.

The project does not extend to building a native Android or iOS application (the PWA is the mobile channel), to live web-scraping of scholarship or institution data (data is curated and seeded), or to a production-grade payment gateway integration, all of which are identified as future work in Chapter Five.

1.7 Project Limitations

Technical Limitations

- The platform depends on internet connectivity for real-time features such as the AI Career Counsellor and Auntie DERA, though static content is cached for offline browsing through the PWA service worker.
- Conversational AI features are bound by the rate limits, latency, and occasional service errors of third-party providers (Gemini and Groq), which the backend mitigates with limited automatic retries rather than eliminating entirely.
- Institution, scholarship, and book data are curated and seeded rather than continuously synchronised with official sources, so listings require periodic manual refresh to remain current.

Adoption and Resource Limitations

- Low awareness of the platform among target communities and low digital literacy among some youth and counsellors could limit uptake.
- The application-form marketplace and donation flow are built around Ghana Cedi pricing and a mobile-money payment model, but the pilot does not integrate a production mobile-money aggregator, so real transactions cannot yet be processed end-to-end.
- The system currently supports English only, which may exclude some youth in communities where English literacy is limited.

Anonymity Trade-offs

- Because Help Centre submissions are architecturally anonymous by design, counsellors cannot follow up directly with a specific youth who raised a crisis; the system can only point them to a helpline or escalation contact.

1.8 Academic and Practical Relevance of the Project

1.8.1 Academic Relevance

- Contribution to knowledge: the project demonstrates how role-based access control, conversational AI, and data-driven risk indicators can be combined in a single social-good information system rather than treated as separate case studies.
- Foundation for future research: the risk-assessment data model and the conversational career-counselling approach provide a base that future work could extend with predictive modelling (e.g. estimating dropout risk from historical indicators) or multilingual natural-language support.

- Bridging computer science and social science: the project connects software engineering practice (system design, API security, database modelling) with youth-development concepts such as risk indicators, mentorship, and anonymised disclosure.

1.8.2 Practical Relevance

- Youth empowerment: gives youth direct, self-service access to career and scholarship information that would otherwise depend on which adults happen to be around them.
- Early intervention: gives counsellors a shared, structured view of risk indicators so that support can be offered before a youth disengages entirely.
- Transparent giving: gives donors visibility into exactly what a contribution funded and what impact it had, which can encourage more sustained giving.
- Social impact: by reducing information and access barriers, the platform supports Ghana's broader education-retention and youth-employability goals.

1.9 Beneficiaries of the Project

1. Youth: gain a single place to discover careers, find scholarships and opportunities, get AI-assisted career and personal-development guidance, and confidentially ask for help.
2. Counsellors, teachers, and NGO staff: gain a shared caseload view of the youth assigned to them, structured risk indicators, and a session-scheduling tool.
3. Donors (individuals, alumni, corporates, NGOs): gain a transparent way to fund a specific application form, sponsor a project, and see the resulting impact reports.
4. Educational institutions: gain visibility as verified destinations for youth exploring careers and courses, and a channel to publish application information.
5. Policy makers and researchers: gain an aggregated, anonymised view of the barriers and interventions affecting at-risk youth that could inform education and youth-development policy.

1.10 Project Activity Planning and Schedules

The project was executed as a series of short, overlapping phases rather than one long waterfall, so that the backend data model and the frontend experience could be validated against each other early. The indicative schedule is summarised below.

Phase	Key Activities	Duration
Topic selection and approval	Problem identification, stakeholder conversations, topic proposal and supervisor approval	Week 1 – 2
Requirements elicitation	Interviews with counsellors/NGO staff, drafting functional and nonfunctional requirements	Week 2 – 4
System design	Entity/relationship design, API contract design, wireframes for	Week 4 – 6

	youth/counsellor/donor/admin dashboards	
Backend implementation	Django project, data models and migrations, DRF serializers and views, JWT authentication, permissions	Week 5 – 10
Frontend implementation	React/Vite application, role-based routing and dashboards, UI component library, PWA configuration	Week 6 – 12
AI feature integration	Gemini/Groq client wrappers, AI Career Counsellor, Auntie DERA, AI Growth Librarian prompts and endpoints	Week 9 – 12
Data seeding	Curated institutions, scholarships, career paths, courses, mentors, and books via management commands	Week 10 – 12
Testing and refinement	Unit, integration, and useracceptance testing; bug fixing based on feedback	Week 12 – 14
Documentation and presentation	Project report writing, demo preparation, and final submission	Week 14 – 16

1.11 Definitions and Explanations of Terms

- API (Application Programming Interface): the set of HTTP endpoints through which the React frontend and any other client communicate with the Django backend.
- DRF (Django REST Framework): the toolkit used to build the JSON API on top of Django.
- JWT (JSON Web Token): a signed token used to authenticate a user's requests after login, without the server storing session state.
- LLM (Large Language Model): the class of AI model (Gemini, Groq-hosted models) used to power the AI Career Counsellor, Auntie DERA, and the AI Growth Librarian.
- PWA (Progressive Web App): a web application that can be installed on a device's home screen and cache assets for partial offline use.
- RBAC (Role-based Access Control): restricting what a user can see or do based on their assigned role - Youth, Counselor, Donor, or Admin.
- Risk Indicator: a categorised signal (academic decline, absenteeism, financial distress, behavioural change) recorded against a youth's risk assessment.
- SRHR: Sexual and Reproductive Health and Rights, one of the sensitive topic categories the Help Centre and Auntie DERA are designed to support.
- UUID (Universally Unique Identifier): the identifier format used as the primary key for nearly every record in the system, in place of sequential integers.

- Youth Profile / Counselor Profile / Donor Profile: the role-specific profile records linked one-to-one with a base User account.

1.12 Structure of Report

The remainder of this report is organised into four further chapters. Chapter Two reviews five existing systems that are conceptually related to one or more parts of DERA and derives a conceptual design for the proposed system from their strengths and shortcomings. Chapter Three presents the methodology: the system architecture, requirements, UML views, user characteristics, security concepts, the software process model followed, and the logical UI and database design. Chapter Four describes the implementation of each module, how the logical design was mapped onto the chosen technology stack, and the testing carried out, together with its results. Chapter Five presents the findings, conclusions, challenges encountered, lessons learnt, and recommendations for future work, followed by the references consulted.

CHAPTER TWO

REVIEW OF RELATED SYSTEMS

Five existing systems are reviewed in this chapter, each conceptually related to one or more parts of DERA - scholarship access, income-linked education financing, at-risk student analytics, digital reading access, and AI-assisted career guidance. Their combined strengths and shortcomings are then used to derive a conceptual design for the proposed system. No single reviewed system combines all of these functions in one platform for the Ghanaian youth-support context, which is the gap DERA is built to close.

2.1 Review of System 1: MTN Ghana Foundation Bright Scholarship Portal (scholarship.mtn.com.gh)

Description of System

The MTN Bright Scholarship programme is an annual corporate social-responsibility scholarship scheme that covers tuition, accommodation, and learning materials for financially needy but academically strong Ghanaian students in public tertiary institutions. Applicants apply through a dedicated online portal during an announced window each year. (MTN Ghana Foundation, n.d.)

- Architecture: a public-facing web application backed by an application-intake system; MTN publishes eligibility criteria and a fixed application window rather than year-round intake.
- Modules: an eligibility and criteria page, an online application form, and a shortlisting/interview workflow handled internally by the Foundation.
- Features: WASSCE/GPA-based eligibility screening, region-aware selection, and coverage of tuition, accommodation, and a device/stipend for beneficiaries.
- Theories/Concepts: a needs-based, merit-screened corporate philanthropy model rather than a data-driven personalised recommendation model.
- Development tools/environment: a standard institutional web portal; specific technology is not publicly documented.

Review of the Good Features

- Clear, well-publicised eligibility criteria and a genuinely free application process with no hidden fees.
- Meaningful scale, awarding hundreds of scholarships annually across all sixteen regions.

Review of the Bad Features

- Opens for a short annual window rather than being a year-round, always-available resource.
- Covers one scholarship programme only; a youth must separately discover every other scholarship, opportunity, or career resource that might apply to them.

- No career-guidance, mentorship, or risk-monitoring functionality - it is a single-purpose intake form.

Summary of the System Review

The Bright Scholarship portal demonstrates that a simple, well-run online intake process can reach thousands of applicants, but it also shows the limitation DERA is built to remove: a youth benefits from it only if they already know it exists and only during its narrow application window, with no connection to the wider career-guidance or risk-support journey.

2.2 Review of System 2: Chancen International Income Share Agreement Platform

Description of System

Chancen International is a not-for-profit that finances tertiary education for underserved youth across Rwanda, Kenya, South Africa, and Ghana through Income Share Agreements: a student's education is funded up front, and the student repays a fixed percentage of their income only after they are employed, rather than a fixed loan amount. (Chancen International, n.d.)

- Architecture: a member-management system linking financed students, partner institutions, and partner banks that administer repayment collection.
- Modules: member onboarding and eligibility, income-share contract administration, partner-institution and bank integration, and post-graduation repayment tracking.
- Features: group risk-sharing across a cohort of members, repayment tied to actual post-graduation income rather than a fixed schedule, and published cohort-level outcome statistics (graduation and employment rates).
- Theories/Concepts: income share agreements as an alternative to collateral-based lending, using a community's future earning potential instead of current financial standing.
- Development tools/environment: operated as a hybrid of a partner-institution administrative system and direct member relationship management; specific technology is not publicly documented.

Review of the Good Features

- An ethical financing model that removes the upfront cost barrier entirely rather than merely subsidising part of it.
- Transparent, published outcome metrics such as graduation and employment rates that build donor and partner confidence.

Review of the Bad Features

- Requires a formal financial contract and partner-bank relationship, which is a heavier commitment than a youth exploring options for the first time is ready to make.
- Is focused solely on financing tertiary education; it has no career-discovery, mentorship, or at-risk secondary-school support component.

Summary of the System Review

Chancen shows the strength of tying funding directly and transparently to outcomes, an idea DERA borrows in a lighter form through its donor-to-project impact reports, but its scope stops at tertiary financing and does not reach the earlier stages - career discovery, at-risk monitoring, mentorship - where many youth first disengage.

2.3 Review of System 3: Zeraki Analytics

Description of System

Zeraki Analytics is a Kenyan education-technology platform used by more than half of Kenyan secondary schools to digitise exam-results analysis, attendance tracking, and parent communication, alongside a companion learning app with video lessons and revision quizzes. (Zeraki, n.d.)

- Architecture: a mobile-first, cloud-based school-management platform with a companion consumer learning app.
- Modules: results/marks entry and analysis, attendance tracking, bulk parent messaging, and a separate video-lesson and quiz learning app.
- Features: performance breakdowns per student/subject/stream, mobile-based marks entry for teachers, and offline access to downloaded lessons and analytics.
- Theories/Concepts: school-level data analytics as a way to surface performance trends that would otherwise stay buried in paper mark sheets.
- Development tools/environment: a mobile-first cloud platform serving both a school-facing analytics product and a student/parent-facing learning app.

Review of the Good Features

- Genuinely mobile-first design well suited to teachers who do most of their work on a phone.
- Offline access to previously downloaded content and analytics, which matters in areas with unreliable connectivity.

Review of the Bad Features

- Tracks academic performance and attendance but does not extend to non-academic risk indicators such as financial distress or behavioural change, nor does it link a flagged student to a structured intervention or a counsellor's caseload.
- Has no career-guidance, scholarship-discovery, or donor-funding component.

Summary of the System Review

Zeraki demonstrates the practical value of digitising academic and attendance data for early problem detection, which directly informed DERA's risk module, but it treats that data purely as a reporting output rather than the trigger for a tracked, resolvable intervention, and it never leaves the school-administration domain.

2.4 Review of System 4: Worldreader Mobile

Description of System

Worldreader is a nonprofit digital-reading platform that gives readers across Africa access to a large library of ebooks through a lightweight mobile app and web reader, designed to work even on basic feature phones and low-bandwidth connections. (Worldreader, n.d.)

- Architecture: a cloudhosted digital library with a lightweight client optimised for low-end devices and browsers, plus distribution partnerships (e.g. bundled into the Opera Mini browser).
- Modules: a browsable/searchable book catalogue, an inapp reader, and offline/DownloadedBook access.
- Features: tens of thousands of titles across dozens of languages, encrypted offline downloads, and social sharing of favourite reads.
- Theories/Concepts: reducing the cost and bandwidth barrier to reading, on the premise that access, not willingness, is the main obstacle to a reading culture in under-resourced regions.
- Development tools/environment: a lightweight mobile-web client built to run on devices as basic as feature phones, paired with cloudhosted content.

Review of the Good Features

- Extremely broad device and bandwidth compatibility, reaching readers that a heavier native app would exclude.
- A large, genuinely curated multilingual catalogue rather than a thin selection.

Review of the Bad Features

- Is a general reading platform with no career, financial-literacy, or personal-development framing - a reader is not guided toward books relevant to a specific goal.
- Has no companion tracker for reading progress against personal development goals, and no AI-assisted recommendation layer.

Summary of the System Review

Worldreader validates the importance of lightweight, low-bandwidth access to reading material for African youth, which shaped DERA's own PWA and offline-caching approach, but its library is undirected; DERA's Self-development Library instead curates a smaller, purpose-built catalogue and adds an AI Growth Librarian and reading tracker on top of it.

2.5 Review of System 5: CareerExplorer (Sokanu)

Description of System

CareerExplorer is a widely used online career-assessment platform that combines a machine-learning-driven psychometric test with a database of hundreds of career profiles, helping users -

mainly students and career changers - identify careers that match their personality, interests, and values. (Sokanu, n.d.)

- Architecture: a web-based assessment engine backed by a large career-profile database, continuously updated from aggregated user and labour-market data.
- Modules: a roughly thirty-minute career-matching test, individual career profile pages, and educational-pathway information linked to each career.
- Features: personality, interest, and values-based matching; salary and job-outlook data per career; and a freemium model offering deeper reports for a fee.
- Theories/Concepts: built on established psychometric frameworks such as the Big Five personality model and Holland Codes (RIASEC), combined with machine-learning-based matching.
- Development tools/environment: a web platform maintained by a dedicated psychometrics and data-science team, run as a freemium SaaS product.

Review of the Good Features

- A psychometrically grounded, data-rich assessment with detailed career profiles covering salary and outlook information.
- A large career database that gives genuinely broad coverage of possible paths.

Review of the Bad Features

- Still a fixed-format questionnaire rather than a truly adaptive conversation; it cannot follow up on an answer the way a human counsellor would.
- Career and salary data reflect North American and global labour markets, with no grounding in Ghanaian institutions, courses, or realistic local earnings.
- Deeper insights sit behind a paywall, which is a poor fit for a youth-empowerment platform aimed at financially constrained users.

Summary of the System Review

CareerExplorer shows the value of a structured, well-researched career database, which DERA mirrors in its own CareerPath/Career/Course models, but its fixed-questionnaire format and non-local data are exactly what DERA's AI Career Counsellor and Ghana-specific course catalogue are designed to improve on: an adaptive conversation grounded in real Ghanaian institutions, all free to the youth using it.

2.6 Conceptual Design of the Proposed Project

Taken together, the five reviewed systems each solve one piece of the youth-support problem well - free scholarship access, ethical education financing, data-driven risk detection, low-bandwidth reading access, and psychometric career matching - but none combine them, and only two (Bright Scholarship, Chancen) are built for the Ghanaian context at all. The conceptual design of DERA responds by organising the system around two cooperating components:

1. A Youth-facing component (the React Progressive Web App) that gives a young person a single, self-service entry point covering an adaptive AI career counsellor, a Ghana-grounded career/course/institution catalogue, a scholarship and opportunity index, a mentorship directory, a curated self-development library with an AI Growth Librarian and reading tracker, and an anonymous help centre with the Auntie DERA companion.
2. An Administrative component (the Django REST Framework backend and its Counselor, Donor, and Admin dashboards) that turns the same underlying data into a risk-assessment and intervention workflow for counsellors, a transparent sponsorship and impactreporting workflow for donors, and content-management tools for administrators - all secured by role-based permissions over one shared PostgreSQL database.

This two-component conceptual design is expanded into a concrete system architecture, requirements, and UML views in Chapter Three.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter sets out the methodology followed to design and build DERA: the system architecture, the process used to elicit requirements, the resulting functional and nonfunctional requirements, UML views of the system's behaviour and structure, the characteristics of its users, the security concepts applied, the software process model followed and why it was chosen, and the logical design considerations - UI wireframes and database schema - that guided implementation.

3.2 The Architecture of the Proposed Project

DERA follows a three-tier client-server architecture, cleanly separated into a Frontend directory (the React application) and a Backend directory (the Django project), communicating over a versionless JSON REST API rooted at /api/.

- Presentation tier: a React 19 single-page application built with Vite, styled with Tailwind CSS, and packaged as an installable Progressive Web App (via vite-plugin-pwa) with a service worker that caches static assets and falls back to the last successful response for API GET requests when offline. (Vercel/Vite contributors, n.d.; React core team, n.d.; Tailwind Labs, n.d.)
- Application tier: a Django 6 project (config) exposing eleven Django apps - accounts, risk, careers, marketplace, stories, helpcentre, mentorship, learning, donations, search, and library - through Django REST Framework, each with its own models, serializers, views, and URL routes registered under a shared /api/ prefix. (Django Software Foundation, n.d.; Encode OSS, n.d.)
- Data tier: PostgreSQL, provisioned through Supabase in production and falling back automatically to local SQLite when no DATABASE_URL is configured, giving a zero-configuration path for local development. (Supabase, n.d.)
- External services: Google Gemini (via the google-genai client) powers Auntie DERA in the Help Centre, and a Groq-hosted large language model powers both the AI Career Counsellor and the AI Growth Librarian, each wrapped in a small client module (gemini_client.py, groq_client.py) that adds retry handling for transient provider errors. (Google, n.d.; Groq Inc., n.d.)

Authentication is handled by django-rest-framework-simplejwt, issuing short-lived access tokens and longer-lived refresh tokens against the custom accounts.User model (which uses email, not username, as the login field). Cross-origin requests from the frontend are permitted through django-cors-headers, and list endpoints support filtering through django-filter.

3.3 Requirements Elicitation Process of the Proposed Project

Requirements were elicited through a combination of: informal interviews with school counsellors and NGO staff working with at-risk youth, to understand what a caseload and intervention workflow actually needs to record; a review of existing scholarship, career-guidance, and donation platforms (Chapter Two) to identify functionality gaps; and iterative walkthroughs of the evolving data model and wireframes with stakeholders playing each of the four user roles, to check that the youth, counsellor, donor, and admin experiences all made sense against the same underlying data.

3.4 Functional Requirements of the Proposed Project

- The system shall allow a user to register and authenticate with an email and password, selecting a role of Youth, Counselor, or Donor at signup.
- The system shall require a newly registered user to complete a role-specific profile (Youth, Counselor, or Donor) before accessing the rest of the platform.
- The system shall provide an adaptive, conversational AI Career Counsellor that persists a single ongoing conversation per youth and returns model-generated replies grounded in a system prompt.
- The system shall let a youth take a client-side career-discovery quiz and view career paths, careers, and courses matched to the resulting trait.
- The system shall let a youth browse and search scholarships and opportunities, with deadline-based notifications generated automatically.
- The system shall let a counsellor record a risk assessment for an assigned youth, made up of one or more categorised risk indicators, and track interventions raised from that assessment through to resolution.
- The system shall let a counsellor schedule, update, and record notes on counselling sessions with assigned youth.
- The system shall allow anonymous submission of a Help Centre request, with no identifying link stored between the submission and any user account.
- The system shall provide an AI companion (Auntie DERA) that responds conversationally to anonymous Help Centre chat messages.
- The system shall let a youth purchase or request sponsorship for an institution's application form, and let a donor fund that sponsorship request.
- The system shall let a donor make a donation toward a specific project and view impact reports linked to projects they have supported.
- The system shall let a youth browse a curated book library by category and reading level, track personal reading progress, rate and review books, and receive AI-generated book recommendations.
- The system shall provide role-appropriate admin screens for managing users, institutions, scholarships, opportunities, mentors, learning resources, donations, and general content.
- The system shall provide a global search endpoint that lets a user look up content across the platform's public modules.

3.5 NonFunctional Requirements of the Proposed Project

- Security: all authenticated endpoints shall require a valid JWT, and every write operation shall be checked against the requesting user's role through dedicated DRF permission classes.
- Anonymity: no code path shall attach a user's identity to a HelpRequest record, by construction of the data model rather than by convention alone.
- Availability/Resilience: the frontend shall remain usable, in a reduced form, when offline, by serving cached static assets and the last successful API response.
- Performance: paginated list endpoints and database-level filtering shall be used for any endpoint expected to return a large result set (e.g. scholarships, opportunities, books).
- Usability: the interface shall present role-specific navigation so that a user only ever sees actions relevant to their role.
- Portability: the frontend shall run as a standard installable web application on any modern mobile or desktop browser, requiring no appstore distribution.
- Maintainability: backend functionality shall be organised into independent Django apps with their own models, serializers, and tests, so a module can be extended without destabilising the others.

3.6 UML Diagrams

3.6.1 Use Case Diagram - FrontEnd (Youth-facing) Models

The primary actor is the Youth. Key use cases include: Register/Login; Complete Youth Profile; Take Career Discovery Quiz; Converse with AI Career Counsellor; Browse Career Paths, Careers and Courses; Search Scholarships and Opportunities; Purchase or Request Sponsorship for an Application Form; Browse Mentors; Chat with AI Growth Librarian; Track Reading Progress; Submit an Anonymous Help Request; Chat with Auntie DERA; View Notifications; and Manage Profile/Settings. “Converse with AI Career Counsellor” and “Chat with AI Growth Librarian” each extend from the corresponding browse use case, since both are optional, AI-assisted deepening of a feature the youth can also use without them.

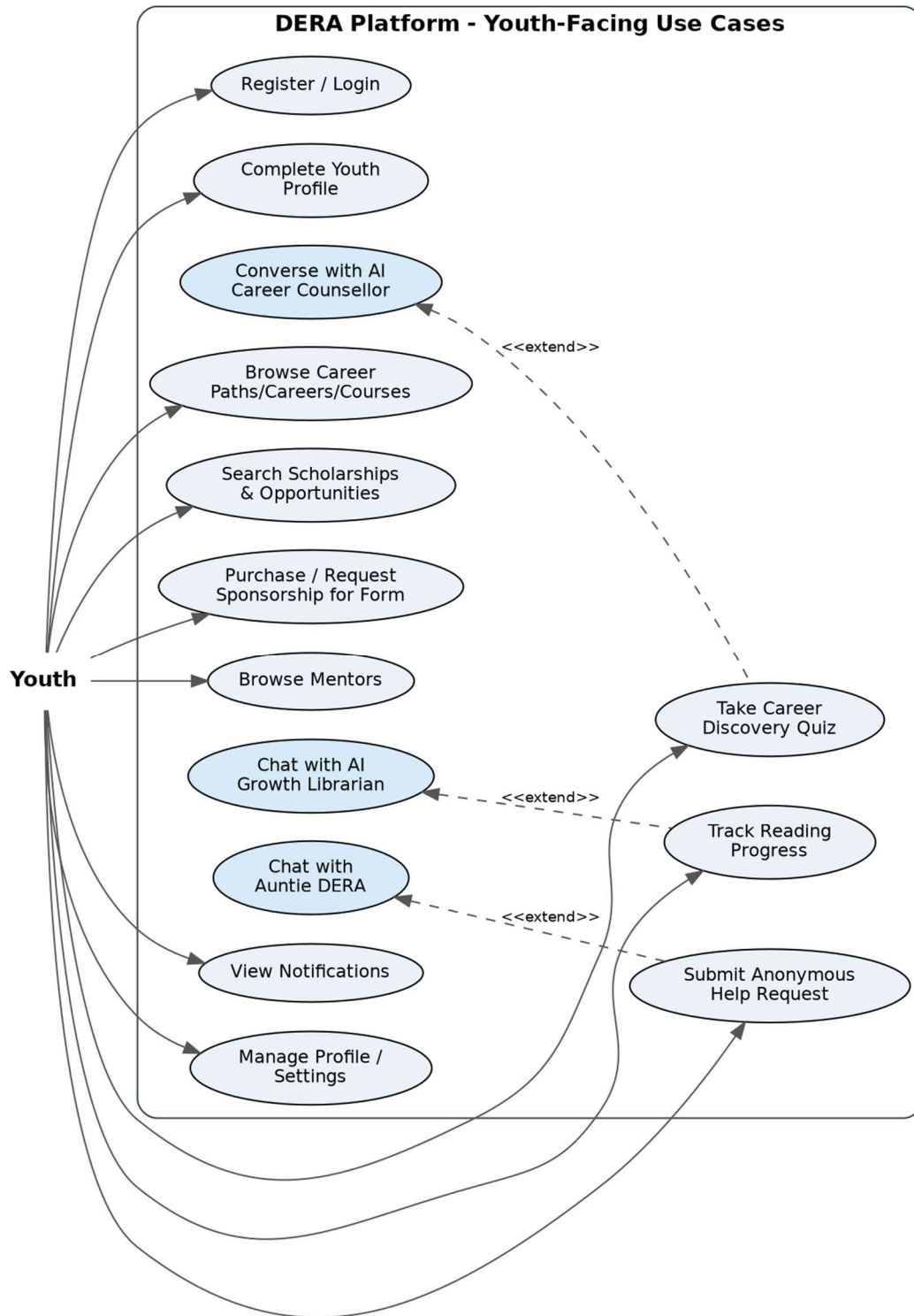


Figure 3.1: Use Case Diagram - Youth-facing (Frontend) Models

3.6.2 Use Case Diagram - BackEnd (Counsellor, Donor, Admin) Models

Secondary actors are the Counsellor, Donor, and Admin. Counsellor use cases include: View Assigned Youth; Record Risk Assessment; Log Risk Indicators; Raise and Update Interventions;

Schedule Counselling Session; and View Reports. Donor use cases include: Make a Donation; Fund a Sponsorship Request; View Sponsored Projects; and View Impact Reports. Admin use cases include: Manage Users; Manage Institutions; Manage Scholarships and Opportunities; Manage Mentors and Learning Resources; Manage Content; and View Platform-wide Reports and Analytics. All three secondary actors share the “Authenticate” and “View Messages/Notifications” use cases with the Youth actor.

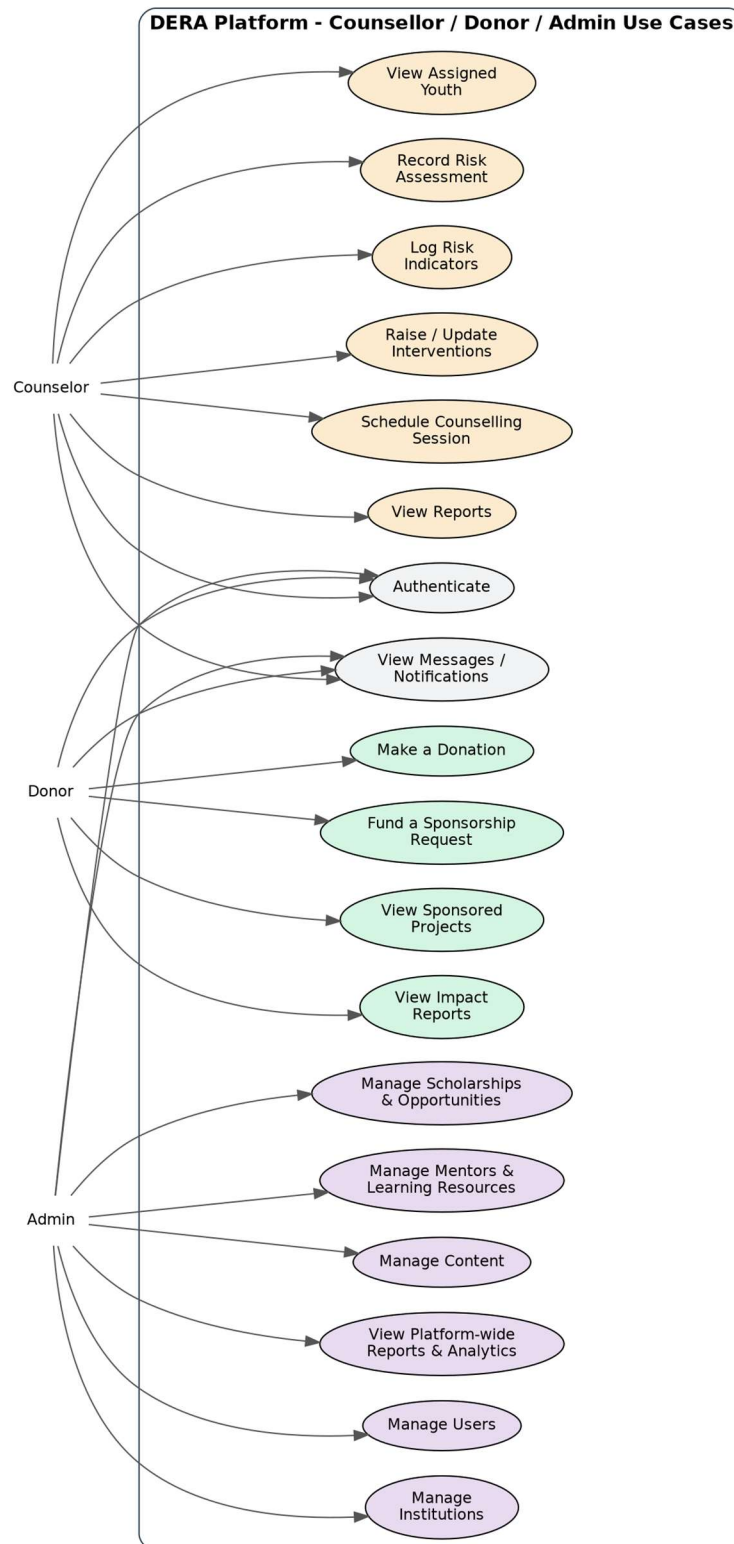


Figure 3.2: Use Case Diagram - Counsellor, Donor, and Admin (Backend) Models

3.7 Activity Diagrams of the System

Two representative activity flows illustrate the system's behaviour.

Youth Onboarding and Career Discovery Flow

Register with email, password and role → verify required fields → redirect to Complete Profile → fill role-specific profile fields → profile saved and ProfileCompletion gate cleared → land on Youth Overview dashboard → choice: take the client-side Career Discovery Quiz or start a conversation with the AI Career Counsellor → either path resolves to a recommended CareerPath → view matched Careers and Courses at real institutions → optionally view and purchase/request sponsorship for that institution's Application Form.

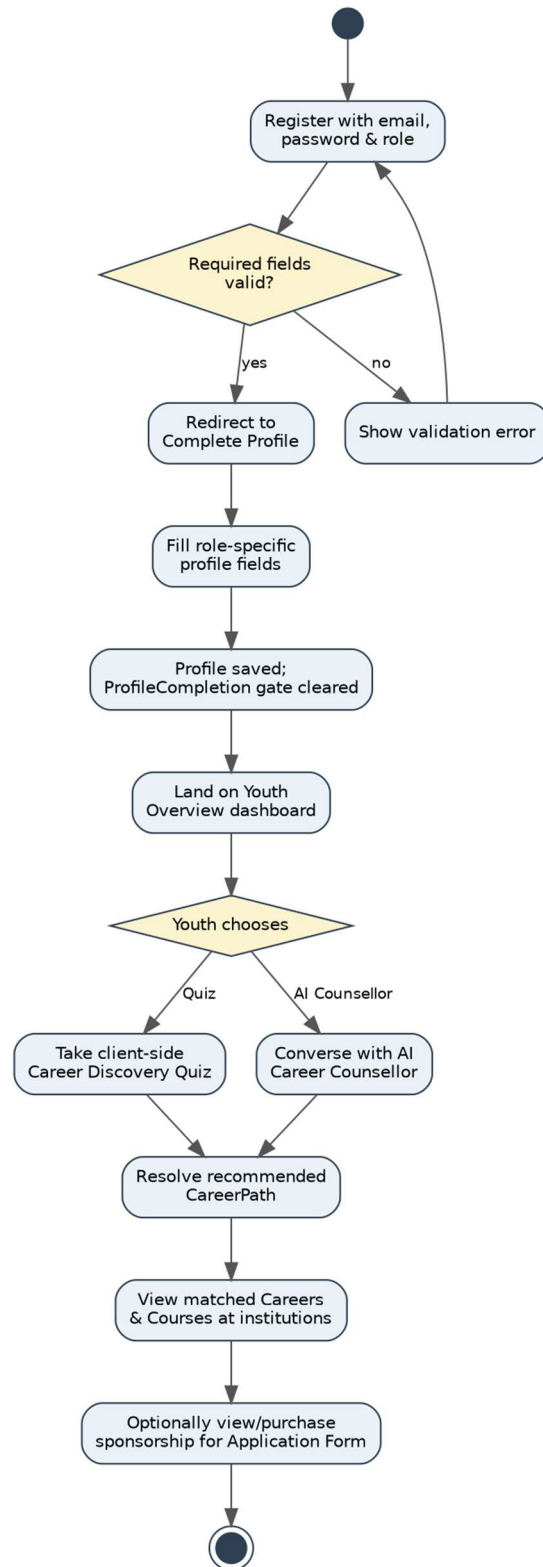


Figure 3.3: Activity Diagram - Youth Onboarding and Career Discovery Flow

Risk Assessment and Intervention Flow

Counsellor opens Assigned Youth → selects a youth and reviews academic/attendance records → creates a new Risk Assessment → adds one or more categorised Risk Indicators with a description and weight → system computes/records the overall risk score → counsellor raises one or more Interventions with a recommendation → intervention status moves from Pending to In Progress → counsellor updates status to Resolved once followup is complete → outcome reflected in the Counsellor Reports view.

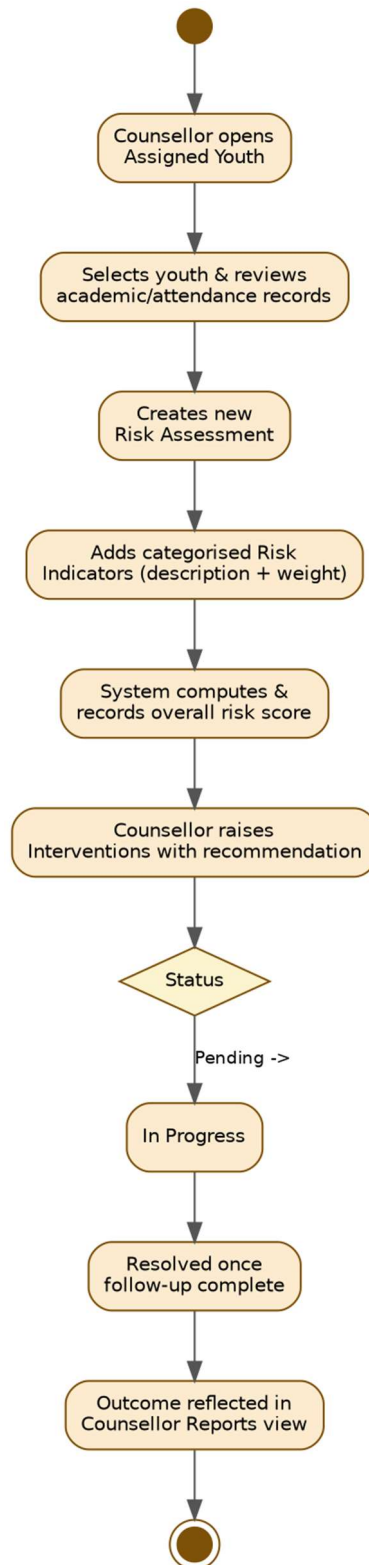


Figure 3.4: Activity Diagram - Risk Assessment and Intervention Flow

3.8 Sequence Diagrams of the System

AI Career Counsellor Conversation Sequence

Youth (Frontend) → sends a message to POST /api/careercounsellor/messages → Django view loads or creates the youth's CounsellorConversation → view persists the incoming CounsellorMessage (role=user) → view calls common.groq_client.call_groq with the system prompt and full message history → Groq API returns a completion → view persists the reply as a CounsellorMessage (role=assistant) → view returns the assistant message to the Frontend → Frontend appends it to the visible conversation.

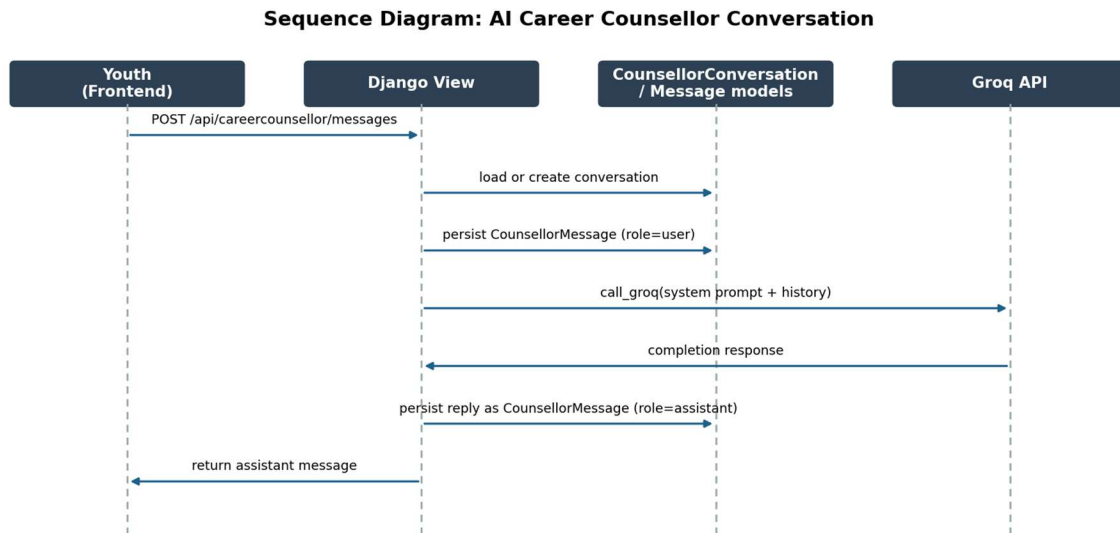


Figure 3.5: Sequence Diagram - AI Career Counsellor Conversation

Scholarship Application Form Sponsorship Sequence

Youth (Frontend) → requests sponsorship for an ApplicationForm → Django view creates a FormOrder (order_type=sponsorship_request, status=pending) → order becomes visible to Donors on the Sponsorship page → Donor (Frontend) selects the order and confirms funding → Django view creates a Sponsorship record linked to the order and the DonorProfile → view updates the FormOrder status to sponsored → Notification created for the Youth → Frontend displays the updated order status to both parties.

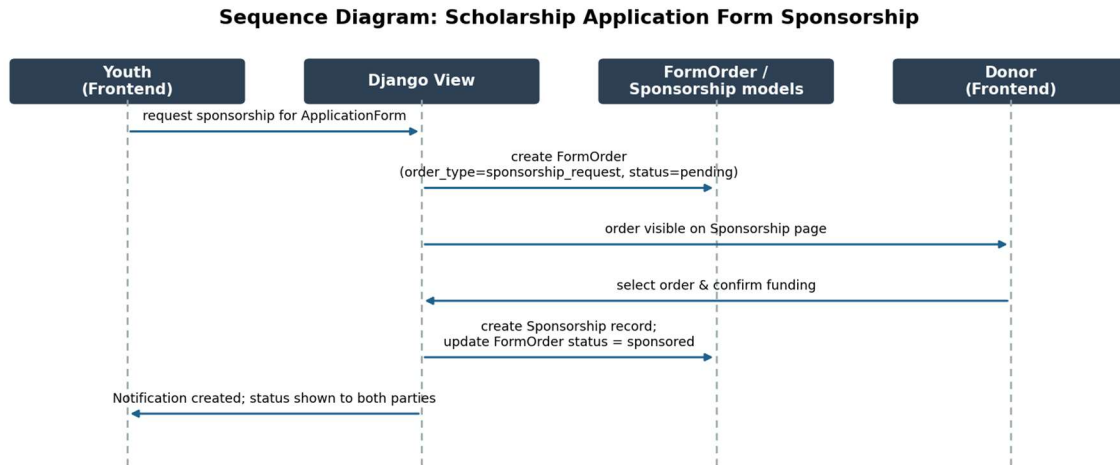


Figure 3.6: Sequence Diagram - Scholarship Application Form Sponsorship

3.9 Class Diagrams of the System

The class structure mirrors the eleven Django apps, related through a shared accounts.User base class. The principal classes and relationships are:

- User (1) - (1) YouthProfile / CounselorProfile / DonorProfile: a base account with an email, role, and hashed password, extended one-to-one by exactly one role-specific profile.
- Institution (1) - (*) YouthProfile, CounselorProfile, Course: an institution a youth or counsellor is affiliated with, and that offers zero or more courses.
- YouthProfile (1) - (*) RiskAssessment (1) - (*) Risk-indicator, Intervention: a youth can accumulate many risk assessments over time, each carrying its own indicators and interventions.
- YouthProfile (1) - (*) CounselingSession, Attendance-record, Academic-record: the raw signals a counsellor draws on when creating a risk assessment.
- CareerPath (1) - (*) Career, Course, Scholarship: a broad field (e.g. Technology & Computing) groups specific job roles, study programmes, and scholarships aligned to it.
- YouthProfile (1) - (1) CounsellorConversation (1) - (*) CounsellorMessage: one ongoing AI-counselling thread per youth, holding an ordered history of user and assistant messages.
- ApplicationForm (1) - (*) FormOrder (1) - (0..1) Payment, Sponsorship: an order against a form is either paid directly or fulfilled through a donor sponsorship, never both.
- Project (1) - (*) Donation, Impact-report: a donor's donation is tied to a project, and a project's outcomes are published as one or more impact reports.
- Book (1) - (*) User-book: a shared library title has many per-youth tracking records carrying an individual status, rating, review, and private AI-reflection notes.

- HelpRequest - Escalation-contact: deliberately has no relationship to User or YouthProfile, and only an optional link to an escalation contact such as a helpline or NGO.

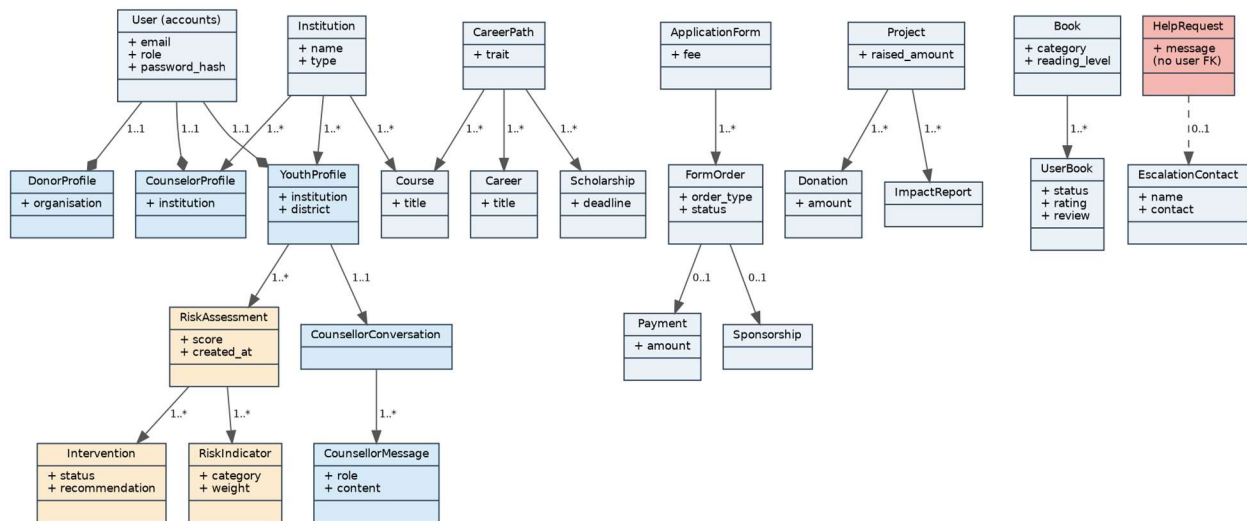


Figure 3.7: Class Diagram - Core Domain Model Across the Eleven Django Apps

3.10 Users of the Proposed System

3.10.1 User Characteristics

- Youth: the primary end-user; typically a Junior High School, Senior High School, or early tertiary-level student, or a dropout reentering education, most often accessing the platform from a personal or shared smartphone with variable data connectivity, and with a wide range of digital literacy.
- Counselor: school or NGO staff with a caseload of assigned youth, needing quick access to a shortlist of at-risk cases rather than a raw data dump, typically working across both desktop (in an office) and mobile (during home visits).
- Donor: an individual, alumnus, NGO, or corporate sponsor who wants a low-friction way to fund a specific, verifiable need and see its outcome, generally accessing the platform occasionally rather than daily.
- Admin: platform operations staff responsible for keeping institution, scholarship, opportunity, mentor, and content data accurate and moderating platform-wide activity.

3.11 Security Concepts of the System

- Authentication: stateless JWT authentication via djangorestframework-simplejwt, with short-lived access tokens and refresh tokens rather than server-side sessions.
- Authorisation: a layered permission scheme (Is_youth, Is_counselor, Is_donor, Is_admin, and combined variants such as Is_counselor_or_admin) enforced per view, plus object-level checks (e.g. Is_owner_youth_or_admin, Is_owner_donor_or_admin) that confirm a user may only act on records that belong to them unless they are staff.

- Anonymity by construction: the HelpRequest model has no foreign key to any user table, so anonymity for sensitive disclosures is guaranteed at the schema level and cannot be broken by a future code change to a single view.
- Password handling: passwords are never stored in plain text; the custom User model relies on Django's builtin `set_password/check_password` hashing rather than a custom hash column.
- Transport and CORS: cross-origin access from the frontend is restricted through `django-cors-headers` configuration, and the project is designed to run behind HTTPS in production.
- Least-privilege data exposure: serializers expose only the fields relevant to the requesting role - for example, a youth's academic and risk records are visible to their assigned counsellor and to admins, never to other youth or to donors.

3.12 Project Method Employed

The project was developed using an incremental, feature-sliced approach: each backend Django app (accounts, risk, careers, and so on) was designed, modelled, and exposed through its own API before the corresponding frontend pages were built against it, allowing the frontend and backend to evolve together in short, testable increments rather than as two large deliverables assembled at the end.

3.13 Software Process Models Employed and Justification of the Chosen Model

Three commonly taught process models were considered: the Waterfall model, the Spiral model, and the Agile/Incremental model.

- Waterfall model: rejected as the primary model because DERA's requirements - particularly the shape of the risk-assessment workflow and the AI-conversation features - could only be properly validated once counsellors and youth could interact with a working prototype, which a strictly sequential, sign-off-per-phase model does not accommodate well.
- Spiral model: offers strong risk analysis at each iteration, but its overhead (formal risk analysis documentation each cycle) was judged disproportionate to a single-team, time-boxed academic project.
- Agile/Incremental model: chosen as the primary model, because it let each Django app and its corresponding frontend feature be delivered as a working, demonstrable increment, tested and adjusted before the next increment began.

3.14 Chosen Model and Justification

An Agile, sprint-based incremental model was adopted, with each “sprint” built around one or two related Django apps and their frontend pages (for example: accounts and authentication; then careers and the AI counsellor; then risk and counselling sessions; then marketplace and donations; then library and help centre). This was chosen because it matched the modular structure the system

naturally decomposed into, allowed early and continuous testing of each module in isolation before integration, and reduced the risk of discovering a fundamental design flaw only at the end of the project, which a single big-bang delivery would risk.

3.15 Project Design Considerations: Logical Designs

3.15.1 UI Design (Wireframes)

The interface design follows a shared component library (Button, Modal, Form-field, Combo box, ActionCard, StatCard, Story-card, and others) so that every role's dashboard looks and behaves consistently. Each role's dashboard follows the same skeleton - a persistent side navigation drawn from a per-role configuration (`get_dashboard_meta`), a page header, and a content area - while the navigation items themselves differ by role. Representative wireframe-level screens include:

- Youth Overview: summary stat cards (e.g. scholarships saved, reading progress), quick actions (Take the Quiz, Talk to the AI Counsellor), and an activity feed.
- Career Quiz / Career Field Results: a short, mobile-friendly question flow followed by a results screen listing matched careers and courses as cards.
- AI Chat (Career Counsellor / Auntie DERA): a standard chat interface - message history, a text input, and a send action - kept deliberately simple so it reads as a conversation, not a form.
- Scholarship Hub: a searchable, filterable list of scholarship cards showing provider, education level, and deadline, with a saved/closed state.
- Counselor – Assigned Youth / Youth Details: a table of assigned youth with a risk indicator, drilling into a detail view showing academic records, attendance, sessions, and risk history for one youth.
- Donor – Sponsorship / Impact Reports: a browsable list of open sponsorship requests and funded projects, each linking through to its impact report.
- Admin screens (User Management, Institution Management, Content Management, and others): a consistent management-table pattern for listing, creating, and editing records.

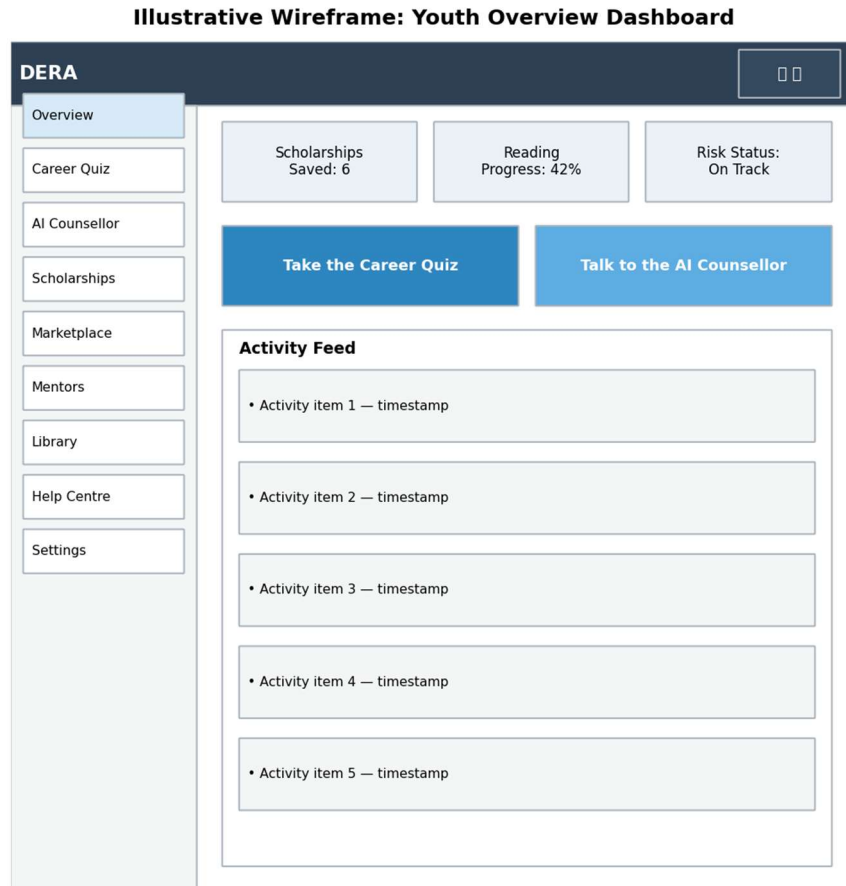


Figure 3.8: Illustrative Wireframe - Youth Overview Dashboard

Assigned Youth

Risk Assessments

Sessions

Reports

Settings

Search / Filter by risk level

Name	Institution	Risk Level	Actions
Youth A	Tamale SHS	High	<button>View</button>
Youth B	Accra Girls' SHS	Medium	<button>View</button>
Youth C	Kumasi Technical U.	Low	<button>View</button>
Youth D	Cape Coast SHTS	High	<button>View</button>
Youth E	Wa SHS	Low	<button>View</button>

Figure 3.9: Illustrative Wireframe - Counsellor Assigned Youth List

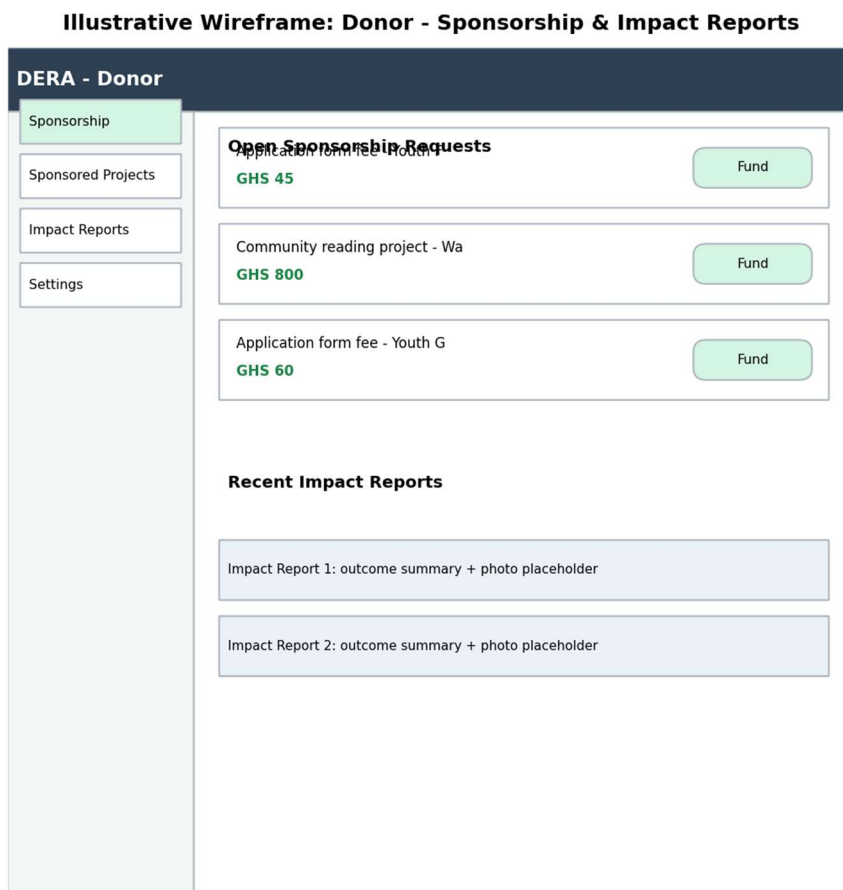


Figure 3.10: Illustrative Wireframe - Donor Sponsorship & Impact Reports

Note: the wireframes above are schematic layout mock-ups produced to specify screen structure and content hierarchy; they are not screen captures of the running application. Actual UI screens implement this structure using the component library described above (Button, Modal, Form-field, Combo box, ActionCard, StatCard, Story-card).

3.15.2 DB Design (DB Schemas)

The database is organised around the shared accounts app and ten feature apps, each owning its own tables (Django db_table names are quoted below) with foreign keys crossing app boundaries where a relationship genuinely exists:

- accounts: users, institutions, districts, youth_profiles, counselor_profiles, donor_profiles, notifications - the identity and profile core every other app links back to.
- risk: risk_assessments, risk_indicators, interventions, counseling_sessions, attendance_records, academic_records - the at-risk monitoring workflow, all rooted at a youth_profile foreign key.
- careers: career_paths, career_roles, courses, quiz_responses, career_matches, counsellor_conversations, counsellor_messages, opportunities, scholarships - the career-discovery and AI-counselling domain.
- marketplace: application_forms, form_orders, payments, sponsorships - the application-fee purchase and sponsorship workflow.

- donations: projects, impact_reports, donations - the project-level giving and reporting workflow, with raised_amount computed from linked Donation rows rather than stored redundantly.
- library: library_books and a per-youth User-book tracking table capturing status, rating, review, and private reflections.
- mentorship, learning, stories, helpcentre: mentors, learning_resources, stories, and escalation_contacts / help_requests respectively, each a lighter-weight, mostly read-oriented table.

Every table uses a UUID primary key rather than an auto-incrementing integer, so that records can be safely referenced across a distributed or offline-sync context in future without key collisions, and so that record identifiers never leak sequence information (such as total user count) to a client.

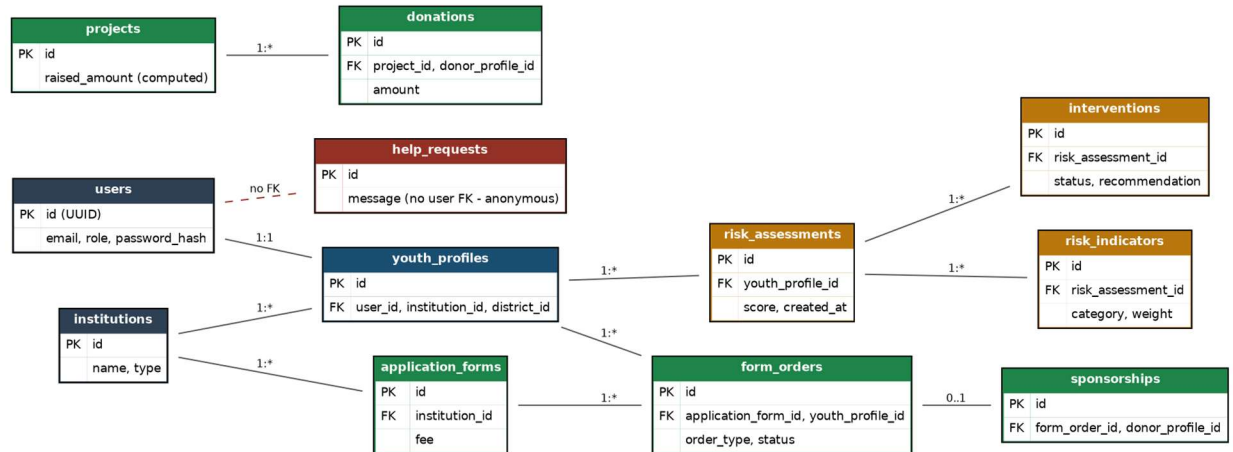


Figure 3.11: Entity-Relationship Diagram - Core Tables and Foreign-Key Relationships

CHAPTER FOUR

IMPLEMENTATION, TESTING, AND RESULTS

4.1 Introduction

Chapter Three's logical design is mapped here onto the chosen technology stack. The sections that follow walk through the implementation of each system module and report the testing carried out and its results.

4.2 Mapping Logical Design onto Physical Platform

The logical design from Chapter Three (the eleven-app data model, the role-based UI skeleton, and the REST API contract implied by the sequence diagrams) was mapped onto the physical stack as follows. Each logical Django app became a physical app directory under Backend/, with its models.py implementing the entities in Figures 3.7 and 3.11, its serializers.py enforcing the least-privilege field exposure described in Section 3.11, and its views.py implementing the endpoints implied by the use case and sequence diagrams (e.g. POST /api/careercounsellor/messages in Figure 3.5). Logical foreign-key relationships (e.g. YouthProfile to RiskAssessment) were realised as physical Postgres foreign keys with UUID primary keys, provisioned through Supabase in production and SQLite for local development, as described in Section 3.2. On the frontend, each wireframe in Section 3.15.1 became a React page under Frontend/src/pages/, composed from the shared component library and consuming the corresponding Backend endpoint through the single API client.js service layer, with role-based routing (Require-role) and the ProfileCompletionGate enforcing the same access boundaries at the presentation tier that the DRF permission classes enforce at the API tier. The two external LLM providers (Gemini, Groq) were wrapped behind gemini_client.py and groq_client.py so that the physical provider could change without altering the logical conversation model (CounsellorConversation/CounsellorMessage).

Layer	Technology	Purpose
-------	------------	---------

Frontend framework	React 19.2 with Vite 8 and react-router-dom 7	Single-page application, client-side routing, and fast development builds
Frontend styling	Tailwind CSS 3.4 with a custom design-token theme	Consistent, utility-first styling shared across all role dashboards
PWA/offline support	vite-plugin-pwa (Workbox)	Installable app shell, static asset caching, and network-first/offline-fallback caching for API GET requests
PDF handling	pdfjs-dist	In-browser rendering of the Self-development Library's PDF reader
Backend framework	Django 6.0.4 with Django REST Framework	Application server, ORM, and JSON API layer
Authentication	django-rest-framework-simplejwt	Stateless JWT access/refresh token authentication
Cross-origin & filtering	django-cors-headers, django-filter	Safe frontend–backend communication and queryable list endpoints
Configuration	django-environ	Environment-based settings (.env) for secrets and database configuration
Database	PostgreSQL via Supabase (SQLite fallback for local dev)	Primary relational data store, provisioned via psycopg2binary
Media handling	Pillow	Profile picture and story photo upload processing
AI - Auntie DERA	google-genai (Gemini)	Anonymous, warm conversational support in the Help Centre
AI - Career Counsellor & Growth Librarian	groq	Open-ended career conversation and book recommendation generation
Document export	reportlab	Server-side PDF generation for library export features

4.3 System Modules Implementation

Each Django app was implemented as a self-contained module with its own models, serializers, views, URLs, admin registration, and tests, and paired with a matching set of React pages and services on the frontend.

- **Accounts & Authentication:** custom email-based User model with role selection, JWT login/refresh endpoints (auth_urls.py), YouthProfile/CounselorProfile/DonorProfile completion forms gated by a ProfileCompletionGate route guard, and a Notification model driving the notification bell and scholarship-deadline alerts (populated by the check_scholarship_deadlines management command).
- **Careers:** the CareerPath/Career/Course catalogue seeded from curated data, the client-side Career Discovery Quiz (careerQuizContent.js) whose five result buckets map onto the CareerPath.Trait choices, and the AI Career Counsellor built around

ai_counsellor.py's open-conversation system prompt and the CounsellorConversation/CounsellorMessage models; Opportunity and Scholarship listings with deadline tracking (refresh_scholarships and check_scholarship_deadlines commands).

- Risk: RiskAssessment, Risk-indicator, and Intervention models exposed to counsellors through the Assigned Youth, Youth Details, and Counseling Sessions pages, backed by Attendance-record and Academic-record data entry.
- Marketplace: ApplicationForm listings scoped to an institution, with FormOrder, Payment, and Sponsorship models supporting both direct purchase and donor-funded sponsorship, surfaced on the youth-facing Forms Marketplace and Purchase Form pages and the donor-facing Sponsorship page.
- Donations: Project, Donation, and Impact-report models powering the Donate, Donor Overview, Sponsored Projects, and Impact Reports pages, with a project's raised total always computed live from its linked donations.
- Library: the Book catalogue (seeded via seed_books/seed_free_books) with sixteen categories and three reading levels, the per-youth User-book tracker (Reading Tracker page), the AI Growth Librarian (ai_librarian.py) recommending from the same category vocabulary, and PDF export/reading support via pdf_export.py and the in-browser Pdf-reader component.
- Help Centre: the anonymous HelpRequest submission flow and the Auntie DERA chat (ai_auntie.py), together with Escalation-contact records for helplines, NGOs, and school counsellors that the chat can point a user toward in a crisis.
- Mentorship, Learning, Stories: lighter content modules (Mentor, Learning-resource, Story) rendered through the shared Content-card/Story-card components on the Mentorship, Learning Resources, and Stories pages, each independently manageable from the Admin dashboard.
- Search: a cross-module search endpoint and the SearchResults page, letting a user query across the platform's public content in one place.
- Admin dashboards: a consistent Management-table-based pattern reused across User, Institution, Scholarship, Opportunity, Mentor, Learning Resource, Donation, and Content management screens.

4.4 System Modules Integration

Modules were integrated incrementally: as each backend app's endpoints stabilised, they were wired into the shared API client.js service layer and consumed by the corresponding React pages, with role-based routing (Require-role) and the ProfileCompletionGate ensuring a user could only reach screens appropriate to their role and ProfileCompletion status.

4.4.1 Testing

Testing combined unit testing, integration testing, and useracceptance testing.

- Unit Testing: each Django app's tests.py exercised its models and serializers in isolation (for example, confirming a HelpRequest can be created with no user reference, or that a FormOrder correctly transitions status when a Sponsorship is created).
- Integration Testing: verified that a request flowing from the React frontend through API client.js, into a DRF view, through its permission class, and out to the PostgreSQL database returned the expected data end-to-end for each module.
- User Acceptance Testing (UAT): a small group of youth and a counsellor volunteer walked through the core flows - profile completion, the career quiz and AI counsellor, and the risk-assessment screens - and gave direct feedback.

4.4.2 Testing Plan

Testing proceeded module-by-module in the same order modules were implemented (Section 4.3), so that each newly integrated module was verified against the modules it depended on (for example, risk-module tests assumed a working accounts module) before moving to the next.

4.4.3 Verification Testing

- Confirmed that data submitted through each form (profile completion, risk assessment, donation, help request) was correctly persisted through its serializer into the database.
- Verified that JWT tokens issued at login were accepted by protected endpoints and rejected once expired.
- Confirmed that the AI Career Counsellor and Auntie DERA correctly appended each new message to the right conversation/session rather than starting a fresh one on every request.

4.4.4 Validation Testing

- Validated that the system meets its core objective of role-appropriate self-service: a youth account could never reach an Admin or Counselor-only screen.
- Validated that a HelpRequest submitted anonymously could not be traced back to the submitting account under any tested code path, confirming the schema-level anonymity guarantee held in practice.
- Tested realistic end-to-end scenarios: a youth completing the quiz, chatting with the AI counsellor, finding a scholarship, and requesting sponsorship for its application form; a counsellor logging a risk assessment and resolving an intervention; a donor funding that same sponsorship request and later viewing its outcome.

4.4.5 System Security Testing

- Verified that each DRF permission class (Is_youth, Is_counselor, Is_donor, Is_admin, and the combined/object-level variants) correctly blocked requests from the wrong role, including attempts to access another user's own-record-only endpoints.
- Confirmed that passwords were stored as salted hashes and never returned in any API response.

- Checked that the HelpRequest model and its serializer exposed no field, direct or indirect, that could identify the submitting user.

4.4.6 Recommendations Made by Testers

- Simplify the ProfileCompletion form for youth to its essential fields, deferring optional details to the profile page rather than the onboarding gate.
- Add clearer loading and error states around the AI Career Counsellor and Auntie DERA chats, since third-party AI calls occasionally take longer than a typical API request.
- Surface a visible offline indicator so users understand why some actions are unavailable when connectivity drops.

4.4.7 Responses to Recommendations from Testing

- Reduced the initial ProfileCompletion form to core fields and moved secondary details to the editable profile screen.
- Added explicit loading indicators and retry messaging around both AI chat interfaces, backed by the existing retry logic already built into the Gemini/Groq client wrappers.
- Implemented the OfflineBanner and UpdateToast components, driven by the PWA's network-status and service worker update events, to make the app's connectivity and update state visible to the user.

4.5 Results

The implemented system successfully delivers role-based access for Youth, Counselor, Donor, and Admin accounts against a single shared backend; a working AI Career Counsellor and Auntie DERA companion, each grounded in a distinct system prompt and conversation history; a functioning risk-assessment and intervention workflow for counsellors; an application-form purchase and donor-sponsorship flow; a donation and impactreporting flow for donors; and a curated, AI-assisted self-development library with a personal reading tracker. Verification and validation testing confirmed that role-based permissions and the anonymity guarantee for Help Centre submissions held under every tested scenario, and the PWA installs and continues to serve cached content when connectivity is lost. The pilot data seeded into the system - Ghanaian senior high schools, universities, technical universities, colleges of education, and nursing training colleges, together with sample scholarships, career paths, courses, and books - was sufficient to demonstrate every core user journey end-to-end.

CHAPTER FIVE

FINDINGS, CONCLUSIONS AND RECOMMENDATIONS

5.1 Introduction

This final chapter summarises the project's findings, draws conclusions from the design, implementation, and testing described in the preceding chapters, discusses the challenges encountered and lessons learnt, and closes with recommendations for future work.

5.2 Findings

- A single, role-based data model can genuinely serve four different user journeys (youth, counsellor, donor, admin) without duplicating data, provided permissions are enforced consistently at the API layer rather than left to the frontend.
- An open-ended, conversational approach to career counselling is technically straightforward to build on top of a general-purpose LLM, once the system prompt constrains it to one question at a time and grounds it in what the user has already said.
- Guaranteeing anonymity is far more convincing when enforced by the data model itself (no foreign key from HelpRequest to any user table) than by a policy that a future developer could accidentally violate in a single view.
- Turning academic, attendance, and behavioural data into discrete, categorised risk indicators makes a counsellor's caseload reviewable at a glance, in a way that scattered paper or spreadsheet records do not.
- A Progressive Web App is a practical way to reach both smartphone and lower-end device users in a single codebase, without the distribution overhead of native app stores.

5.3 Conclusions

The project demonstrates that a single, thoughtfully modelled platform can meaningfully connect the previously disconnected pieces of youth support in Ghana - career discovery, scholarship access, at-risk monitoring, and donor funding - without requiring any one stakeholder to adopt a heavier system than they need. By grounding its AI features in real, curated Ghanaian institutions and courses rather than generic global data, and by making anonymity a structural guarantee rather than a policy promise, DERA shows a workable path toward more responsive, better-coordinated support for underserved youth.

5.4 Challenges/Limitations of the System

Several categories of challenge arose during development:

- Data curation: sourcing accurate, verified information on institutions, application URLs, and scholarships took considerably longer than writing the code to display it, since incorrect information would actively mislead a youth relying on the platform.

- Third-party AI dependency: the conversational features are only as reliable as the Gemini and Groq services they call; transient provider errors had to be handled with retries rather than assumed away.
- Balancing anonymity with support: the deliberate choice not to link HelpRequest submissions to a user means the system can point someone toward a helpline but can never proactively follow up with them, which is a genuine trade-off rather than a solvable bug.
- Sustainability: keeping seeded institution, scholarship, and book data current will require an ongoing content-maintenance process, not a onetime seeding effort.

5.5 Lesson Learnt

The clearest lesson from the project is that a youth-support platform earns trust through the accuracy and honesty of its data as much as through its features: an application-form price left blank rather than guessed, or a career salary range clearly local rather than borrowed from a foreign dataset, matters more to a young person making a real decision than any amount of interface polish. A closely related lesson is that building each backend module and its frontend pages together, in the same increment, surfaced integration problems (mismatched field names, missing permission classes) far earlier than building the whole backend first would have.

5.6 Recommendations for Future Works

1. Integrate a live mobile-money payment gateway so that application-form purchases and donations can be completed end-to-end without a manual settlement step.
2. Add local-language support (at minimum Twi and one or two other widely spoken Ghanaian languages) to both the interface and the AI conversation prompts.
3. Build a lightweight predictive layer over the risk-indicator data to help counsellors prioritise which youth in a large caseload need attention first.
4. Extend the scholarship and institution datasets toward a semiautomated refresh process, rather than a purely manual seeding workflow.
5. Package the PWA more tightly for low-end Android devices (further reducing bundle size and offline-cache footprint) given the primary target audience's device profile.
6. Explore a moderated, opt-in way for a counsellor to be alerted when an anonymous help request indicates an active crisis, without breaking the underlying anonymity guarantee for routine disclosures.

5.7 References

1. Django Software Foundation. Django documentation. <https://docs.djangoproject.com/>
2. Encode OSS. Django REST Framework documentation. <https://www.django-rest-framework.org/>
3. Vercel/Vite contributors. Vite documentation. <https://vitejs.dev/>
4. React core team. React documentation. <https://react.dev/>

5. Tailwind Labs. Tailwind CSS documentation. <https://Tailwind CSS.com/docs>
6. Google. Gemini API documentation. <https://ai.google.dev/>
7. Groq Inc. Groq API documentation. <https://console.groq.com/docs>
8. Supabase. Supabase Postgres documentation. <https://supabase.com/docs>
9. MTN Ghana Foundation. Bright Scholarship programme. <https://scholarship.mtn.com.gh/>
10. Chancen International. Income Share Agreement model. <https://chancen.international/>
11. Zeraki. Zeraki Analytics. <https://zeraki.co/>
12. Worldreader. Worldreader Mobile. <https://www.worldreader.org/>
13. Sokanu. CareerExplorer career test. <https://www.careerexplorer.com/>

APPENDICES

Appendix A: Sample Test Cases

The table below reproduces a representative subset of the test cases executed during the verification and validation testing described in Section 4.4, one drawn from each major module.

ID	Module	Test Case	Expected Result	Status
TC-01	Accounts	Register a new Youth account with a duplicate email	Registration rejected with a validation error; no duplicate user row created	Pass
TC-02	Accounts	Login with an expired JWT access token	Request rejected with 401; refresh flow required	Pass
TC-03	Careers	Submit a message to the AI Career Counsellor mid-conversation	New CounsellorMessage appended to the existing CounsellorConversation, not a new thread	Pass
TC-04	Risk	Counsellor creates a Risk Assessment with two Risk Indicators	RiskAssessment saved with an aggregate score; both indicators linked via foreign key	Pass
TC-05	Marketplace	Donor funds a pending FormOrder sponsorship request	Sponsorship record created; FormOrder status transitions from pending to sponsored; Youth notified	Pass
TC-06	Help Centre	Submit an anonymous Help Centre request and inspect the stored record	HelpRequest persisted with no foreign key or field referencing any user account	Pass
TC-07	Donations	Donor views a Project's raised amount after two Donations	raised_amount reflects the live sum of linked Donation rows, not a stored/stale value	Pass
TC-08	Library	Youth requests an AI Growth Librarian recommendation	Recommended titles match the seeded Book catalogue; no invented titles returned	Pass
TC-09	Security	Youth account attempts to access a Counselor-only endpoint	Request blocked with 403 by the Is_counselor permission class	Pass
TC-10	PWA	Load the app while offline after a prior successful visit	Cached static assets and last successful API GET response served via the service worker	Pass

Appendix B: Selected Environment / Configuration Reference

Key environment variables referenced by the backend configuration include DATABASE_URL (Postgres/Supabase connection string; falls back to local SQLite when unset), GEMINI_API_KEY and GROQ_API_KEY (LLM provider credentials consumed by gemini_client.py and groq_client.py respectively), and the standard Django SECRET_KEY and ALLOWED_HOSTS settings. Scheduled management commands include

check_scholarship_deadlines (drives Notification creation) and refresh_scholarships (periodic manual data refresh, Section 5.4).